EEE3096S Practical 1B

Phila Haanyama
HNYPHI001

Lulama Lingela
LNGLUL002

I. Introduction

In this practical, the STM32F0 microcontroller was tested to see how well it could handle running a computationally heavy task by using a Mandelbrot set algorithm. The Mandelbrot set is a type of mathematical fractal defined in the complex plane and is generated via iterative calculations that can be computationally extensive.By implementing this set algorithm on the STM32F0, this practical allowed for the measurement of execution time and calculation of a checksum which provided a means of testing the microcontroller's processing speed.

Two versions of the algorithm was written: one using fixed-point arithmetic (only integers, with a scaling factor to represent decimals) and one using double-precision floating-point numbers. How long each version took to run for different image sizes was then measured and compared to reference values generated in Python. This gave a clear picture of the trade-offs between using fixed-point and floating-point maths on a low-powered microcontroller.

II. Methodology

1) **Setting up the project**
   Cloned the provided GitHub repository and imported the *Practical_1B* project into STM32CubeIDE.
2) **Adding global variables**
   Declared global variables to store execution timing, checksum results, and the set of image dimensions to test (128, 160, 192, 224, 256).
3) **Fixed-point implementation**
   Implemented `calculate_mandelbrot_fixed_point_arithmetic()` using only integers and a scale factor of 10^6 to represent decimal values.
   Used 64-bit integers for intermediate calculations to prevent overflow. Utilising fixed-point arithmetic instead of just using integers is necessary in order to maintain all fractional precision in coordinate mapping. The boundaries of the Mandelbrot set require fractional resolution to determine the iteration counts of escapes accurately.
4) **Timing and debugging**
   Used `HAL_GetTick()` to record start and end times around the Mandelbrot function, calculating execution time in milliseconds.
   Monitored the `execution_time` and `checksum` variables for each image size in the STM32CubeIDE Live Expressions window.
5) **Double-precision algorithm**
   Implemented `calculate_mandelbrot_double()` using double-precision variables for the same algorithm.
   Measured execution time and checksum again for all image sizes.
6) **Comparing with Python**
   Ran the provided Python script to obtain reference checksum values.
   Compared the Python results with the STM32 outputs for both algorithms.
7) **Recording results**
   Tabulated all timing and checksum values.
   Noted any differences between the fixed-point and double-precision algorithms.

III. Results & Discussion

Image Size	Time (s)	Checksum
128×128	20.917	430527
160×160	32.648	671374
192×192	47.025	967630
224×224	64.038	1317555
256×256	83.549	1718118

TABLE I
Fixed-point execution times and checksums.

Image Size	Time (s)	Checksum
128×128	121.167	429384
160×160	190.458	669829
192×192	274.594	966024
224×224	374.223	1314999
256×256	485.450	1715812

TABLE II
Double-precision execution times and checksums.

Image Size	Time (s)	Checksum
128×128	0.032	429384
160×160	0.072	669829
192×192	0.071	966024
224×224	0.095	1314999
256×256	0.123	1715812

TABLE III
Python execution times and checksums (reference).

The execution time for the Mandelbrot methods increased as the image size increased. When comparing the two different implementations on the STM32F0, using double-precision took consistently approximately 5–6× longer than fixed-point (Figure 1). This delay was expected, as the STM32F0 does

not have a floating-point unit (FPU) and therefore emulates floating-point operations in software, whereas fixed-point arithmetic can utilise native integer operations.

There is a large performance gap between the microcontroller and the laptop (M4 MacBook Pro) processor. This is observed when completing the largest image: STM32F0 required 83.5 s (fixed-point) and 485.5 s (double), while the PC took only 0.12 s (Tables I, II, and III). This difference arises due to factors such as clock speed, memory bandwidth, and the presence of hardware FPUs on the PC.

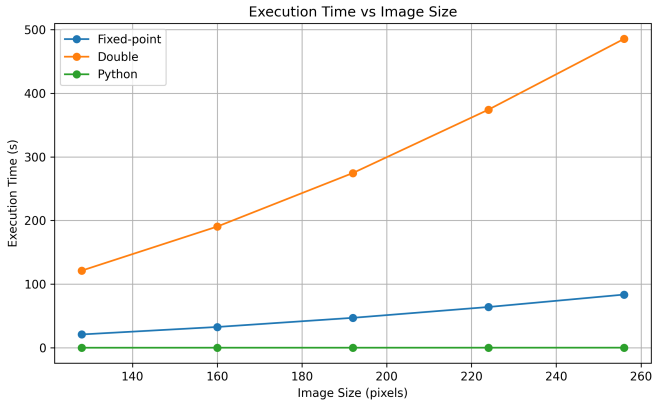

Fig. 1. Execution time vs image size for Fixed-point, Double, and Python implementations.

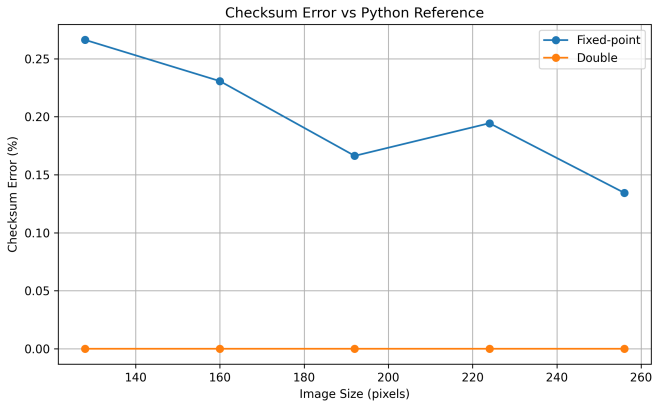

Fig. 2. Percentage checksum error vs image size compared to Python reference.

The checksum value was calculated as the total number of iteration counts for all the pixels in the generated Mandelbrot set image. It acts as a fingerprint of the final output - if two runs give the same checksum, it means the images are identical.

Using double-precision produced identical checksum results to the Python reference for all image sizes, whereas fixed-point deviated by at most 0.27%. This is due to rounding and truncation errors that occur during fixed-point coordinate mapping and iterations. However, the effect of these differences in individual pixel iteration counts is minimal, as the fixed-point implementation has a tolerance within $\pm 1\%$ as specified in the brief.

Image Size	Fixed-point Error (%)	Double Error (%)
128×128	0.2663	0.0000
160×160	0.2309	0.0000
192×192	0.1661	0.0000
224×224	0.1940	0.0000
256×256	0.1342	0.0000

TABLE IV
PERCENTAGE ERROR IN CHECKSUM COMPARED TO PYTHON REFERENCE.

The checksums from the double-precision implementation are more accurate than the fixed-point results. However, there is a trade-off: the fixed-point method runs much faster on the STM32F0, as it does not have a floating-point unit (FPU), which makes integer operations much faster than emulating floating-point operations. Utilising single-precision floats (32-bit) instead of doubles (64-bit) may run slightly faster than the current double-precision method, but will be less accurate due to increased rounding errors.

IV. Conclusion

This practical involved comparing the performance and accuracy of fixed-point and double-precision Mandelbrot implementations on the STM32F0 microcontroller, and comparing the results with a Python script reference running on a PC. The benchmark results showed that fixed-point arithmetic has an approximately 5–6× faster execution time on the STM32F0 in comparison to double-precision, which is slower due to the lack of a hardware FPU. However, double-precision produces accurate results that align exactly with the Python reference output, while fixed-point has a minimal accuracy loss (within $\pm 1\%$ tolerance). The choice between the two depends on whether execution speed or exact numerical accuracy is more important for the application.

For future improvements, using an STM32F4 would enhance the performance of running the Mandelbrot algorithm compared to using an STM32F0. The STM32F4 includes an FPU and has a higher clock speed, which would optimise the execution time for both the fixed-point and double-precision implementations.

V. AI Clause

- Gaining a better understanding about the task through high-level explanations.
- Getting detailed explanations of errors during coding.
- Generated Python code for graphing recorded execution time and checksum results.
- Used to debug LaTeX formatting issues.
- Improving clarity & readability of the report.